

8. El Ordenador

(Edición 2006)

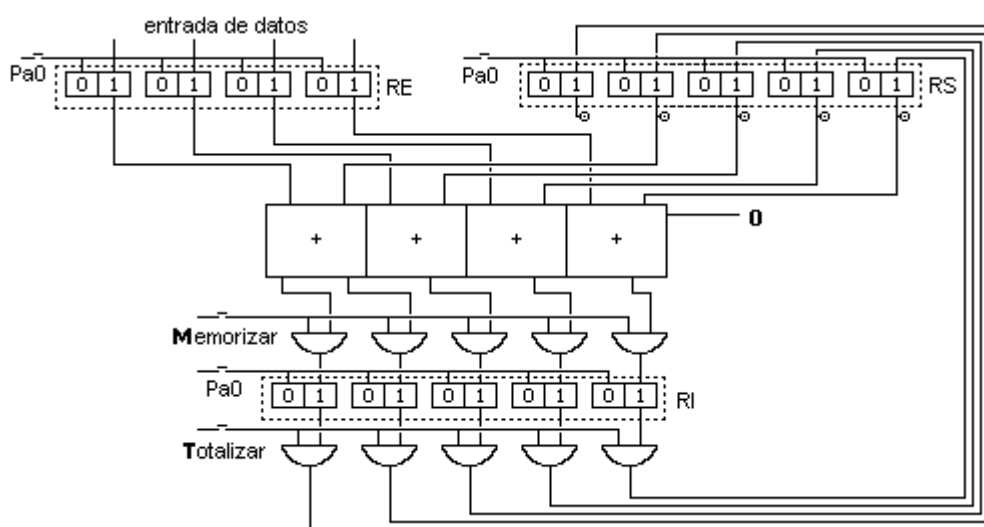
MAQUINA DE PROGRAMACIÓN FIJA

Suma manual.

Con los recursos vistos hasta el momento se puede encarar el diseño de un circuito que nos sirva para ir sumando sucesivas cantidades en un registro que denominaremos ACUMULADOR.

Para ello, necesitamos crear, en primer término un registro (grupo de n biestables) que nos permitan memorizar una cantidad de entrada ingresada por pulsadores, un registro de salida o ACUMULADOR donde quedará el resultado de la suma de las sucesivas entradas y registros intermedios donde depositaremos temporariamente la información. Además y fundamentalmente necesitaremos de un grupo de sumadores que nos permitan realizar efectivamente la operación de suma.

Con estos elementos construimos el siguiente circuito:



Referencias

RE reg. entrada
RI reg. intermedio
RS reg. salida

secuencia de proceso

Puesta a 0 general
0 Ingresa dato en RE
1 Memoriza
2 Puesta a 0 RS
3 Totalizar
4 Puesta a 0 RI y RE
5 Ir a 0

Figura 8-1.

Al iniciar el proceso se hace una puesta a 0 general, pulsando la tecla correspondiente, en cada uno de los registros.

Luego ingresamos en el RE el dato a sumar. Si observamos el circuito, a medida que ingresemos el valor, se va a producir la suma con el contenido del RS (recordar que los sumadores son sistemas combinatoriales y por lo tanto cambian, en la medida que cambian las variables de entrada).

Una vez que se ingresó el valor de entrada completo, y el resultado de la suma esté estable, se debe memorizar en un registro intermedio (RI).

Este paso nos permitirá hacer una puesta a 0 del registro de salida, (que está también involucrado en la entrada) antes de transferir el nuevo resultado.

Digitando la tecla totalizar, se habilita el paso de la información desde el registro intermedio al registro de salida. Y luego, se puede hacer la puesta a cero del registro de entrada y el registro intermedio.

De esta forma, el resultado queda en el registro de salida, y pasa a ser un operando, junto con un nuevo valor que se ingresa en el registro de entrada de la siguiente suma.

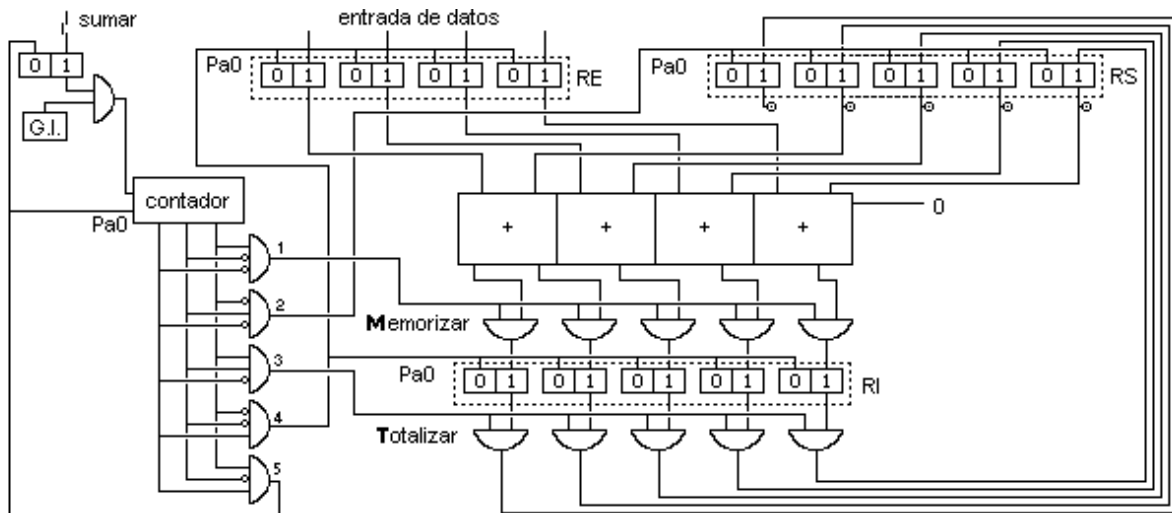
Repitiendo este proceso, se pueden ir acumulando sucesivas cantidades. Aunque, el proceso es demasiado pesado debido a que por cada suma, se deben digitar varias teclas y, en una secuencia determinada.

Automatización del proceso.

Encaremos el diseño tratando de automatizar el proceso. Para ello, agregamos un contador que se incrementa mediante un generador de pulsos, y cada combinación de salida habilita una función. De esta forma logramos que las mismas se cumplan en una secuencia determinada.

Al digitar **suma**, se habilita el paso de pulsos desde el **G.I.** al contador y cuando este vale 1, habilita la tecla memorizar, cuando vale 2, hace la puesta a 0 de RS, al valer 3, totaliza y 4 pone a 0 el RE y RI. El contador en 5 finaliza el proceso, para ello restaura el valor del mismo a cero, y cierra el paso de pulsos, a la espera de un nuevo valor. A estos sucesivos pasos los denominamos microinstrucciones.

De esta forma, nuestra tarea será, ingresar los datos en RE, y digitar **suma**.



- 0 Puesta a 0 general.
 0 Ingresar dato en RE
 1 Dígita **suma**

Figura 8-2.

MAQUINA DE PROGRAMACIÓN VARIABLE

Introducción:

Las máquinas que hemos visto anteriormente, se encasillan en el tipo **de máquinas de programación fija** y responden al esquema siguiente:

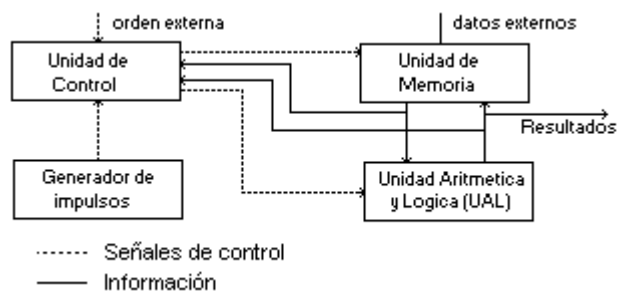


Figura 8-4.

La Unidad de Memoria (U.M.) almacena los datos externos, los resultados parciales y los finales del proceso.

Esta puede estar constituida por algunos registros, tal es el caso de las máquinas vistas o, si el número de datos es elevado (mas de diez), poseer una organización o estructura como las estudiadas en el tema relativo a memorias.

La Unidad de Aritmética y Lógica (U.A.L.) recibe información procedente de la Unidad de Memoria, ejecuta los cálculos y le devuelve los resultados.

La Unidad de Control (U.C.) gobierna ambas unidades.

Este es un sistema secuencial que a partir de una orden externa de inicio de proceso, efectúa una evolución entre estados, desde uno inicial, según un determinado diagrama de flujo, controlado por los impulsos de un generador.

Por cada estado, la U.C. emite una señal de control para que la Unidad de Memoria y la Unidad Aritmética y Lógica realicen la microoperación adecuada. Una microoperación es entonces:

"Una operación Aritmética o Lógica o una transferencia entre registros que se realiza durante un ciclo del generador de impulsos equivalente a un único estado de la U.C."

LA COMPUTADORA.

Arquitecturas Secuenciales.

En esta la U.C., en lugar de recibir una simple orden externa de operación, recibe una "Secuencia De Instrucciones" o "programa almacenado" que le indican las sucesivas microoperaciones a realizar. Las instrucciones deben alojarse en una unidad de memoria.

Esta estructura general, fue definida por Von-neuman en 1946 y en la actualidad rige en el funcionamiento de diferentes modelos y marcas.

El esquema general de bloques que corresponde a la máquina de programación variable es entonces:

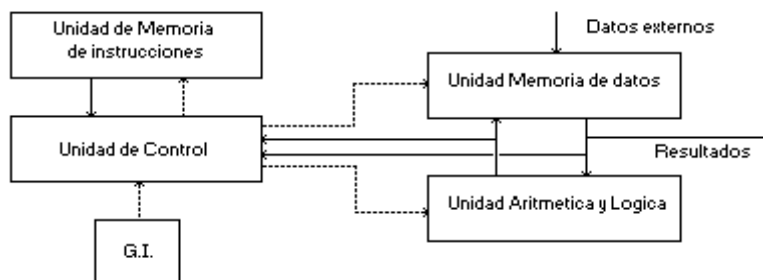


Figura 8-5. Esquema básico de bloques de una máquina de programación variable.

Siendo las instrucciones una combinación de bits que indican a la U. de C. las acciones que debe realizar, pueden entonces almacenarse en la misma U. de M. en que se almacenan los datos. Con lo que el esquema se reduce a:

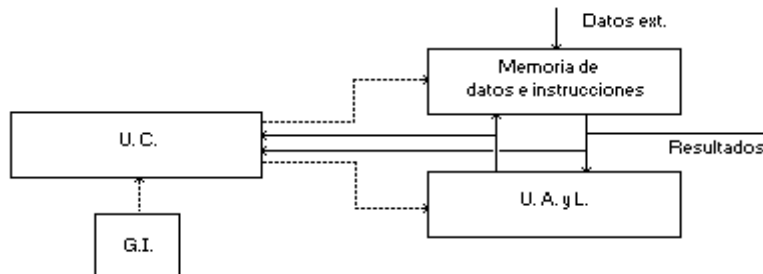


Figura 8-6. Esquema básico de bloques de una máquina de Programación variable con Unidad de Memoria banalizada.

La Unidad de Memoria más conveniente es la de Acceso Aleatorio. Debe poseer un sector activo (Lectura/Escritura) para alojar:

- * Datos
- * Resultados parciales o finales
- * Instrucciones

Como se ha visto en el tema correspondiente a 'Memorias', esta puede además, poseer un sector pasivo (Lectura solamente) para alojar tanto datos como instrucciones fijos.

El conjunto constituido por la U.de C. y la U.A.y L. recibe, por lo general el nombre de Unidad Central de Proceso y La Memoria de datos e instrucciones recibe el nombre de memoria central o principal.

Los datos del proceso se han de transferir desde el exterior. Los resultados del proceso se han de transferir hacia el exterior.

Además las Instrucciones se almacenan en memorias externas y deben ser enviadas a la M.C. en forma similar a los datos.

Los demás elementos o sistemas externos a la U.C.P., M.C. y G.I. se denominan en forma general "PERIFÉRICOS".

De lo antes dicho se deducen dos esquemas generales:

1. Si las transferencias de información se realizan a través de la Unidad Central de Proceso (U.C.P.).

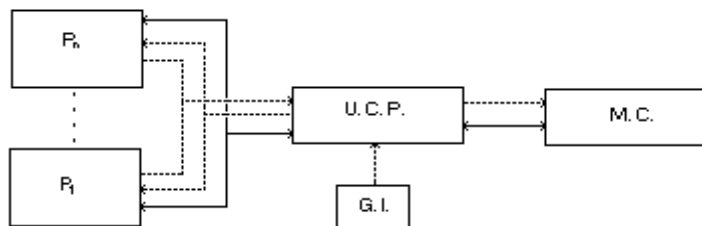


Figura 8-7. esquema, U.C.P. / M.C.

2. Si no se realiza simultáneamente transferencia de información con M.C. y un periférico pueden emplearse conexiones comunes a la M.C. y Periféricos:

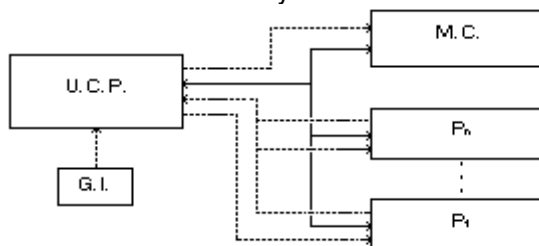


Figura 8-8. esquema, U.C.P. / M.C. y Periféricos

Este es el diagrama o esquema de bloques típico de un Computador u Ordenador.

La Instrucción.

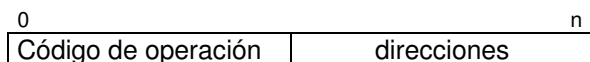
Como dijimos, las instrucciones están constituidas por una combinación de bits que se alojan en palabras de la M.C. y que deben poseer información sobre:

1. Que operación debe realizar el sistema.
2. Con qué operandos realizarla.

Por lo tanto la Instrucción debe poseer dos campos:

- * Campo de código de operación
- * Campo de direcciones de operandos

Estos se esquematizan de siguiente manera:

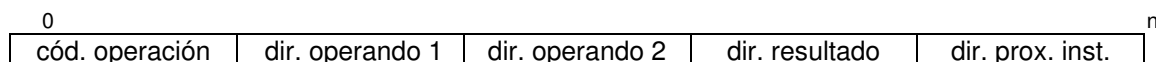


El registro de Instrucciones: La ejecución de una instrucción, ya hemos visto, hace necesario que la Unidad de Control primero "busque" la instrucción en la Memoria Central, para que a continuación pase a la etapa de la ejecución propiamente dicha. Se requiere por lo tanto alojar la instrucción, durante el tiempo que dura su ejecución en un registro ubicado en la U.C.P. que toma la denominación de REGISTRO DE INSTRUCCIONES.

La instrucción de cuatro direcciones: La ejecución de una instrucción implica alguna de las siguientes acciones:

- * La realización de una operación con un solo operando, (por ejemplo "complementar") o dos operandos (por ejemplo suma aritmética) y por ende la necesidad de conocer las direcciones de uno ó 2 operandos.
- * La presencia de un resultado y entonces la necesidad de conocer la dirección donde debe ser alojado.
- * La dirección de la próxima instrucción a ejecutar.

Entonces la forma más simple de informar a la U.de C. es constituir una instrucción con un campo de dirección conteniendo 4 direcciones:

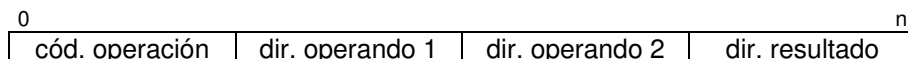


El inconveniente de este formato es que la instrucción necesita un elevado número de bits.

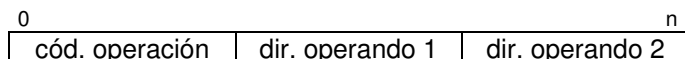
El Contador de Programa: En general las instrucciones se ejecutan secuencialmente. Pueden entonces alojarse en posiciones contiguas de la memoria y por lo tanto la dirección de la próxima instrucción puede sustituirse por un registro contador que se denomina: CONTADOR DE PROGRAMA y cuyo contenido se incrementa en una unidad cada vez que se ejecuta una instrucción. De esta forma

luego de "rescatar" la instrucción corriente y al incrementarse en 1 queda apuntando a la próxima instrucción a ejecutarse. Es importante recordar que esta operación se realiza LUEGO de haber obtenido la instrucción corriente y ANTES de proceder a ejecutar la misma.

El contador de programa posibilita una instrucción de 3 direcciones:



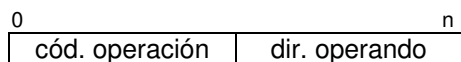
Registro auxiliar: Si se conviene en almacenar el resultado en la dirección de uno de los operandos se logra la instrucción de 2 direcciones:



Hace falta entonces prever una instrucción de Transferencia de Información de una posición de memoria a otra para el caso en que sea necesario salvar o conservar el operando en cuya dirección debe alojarse el resultado.

Además implica la presencia de un Registro Auxiliar en la U.C.P. para conservar momentáneamente las informaciones procedentes de la memoria.

Se puede reducir aún más el número de bits de una instrucción, empleando un solo campo de direcciones con lo que se logra, la instrucción de una dirección:



El registro acumulador: Dado que la instrucción de una dirección sólo selecciona un operando en memoria, hace necesario el empleo en la U.C.P. del registro auxiliar anteriormente mencionado para conservar el otro operando. Este mismo registro suele emplearse además para almacenar el resultado de la operación, tomando por esta causa el nombre de Registro Acumulador.

Muchas computadoras tienen una cierta cantidad de registros de propósito general, cualquiera de los cuales puede ser usado indistintamente como acumuladores. Si por ejemplo hubiera 8 registros de este tipo, se necesitarían solo 3 bits para direccionarlos, lo que es considerablemente menor que la cantidad de bits necesarios para direccionar una palabra de memoria. De esta forma las instrucciones podrían estar conformadas con una dirección de operando de memoria y otra de registro.

Además de los registros que se han visto podemos nombrar aparte: "registro de instrucciones" (IR) contiene la instrucción que está siendo ejecutada cuya salida está disponible para los circuitos de control, los que a su vez generan el secuenciamiento de señales necesario para controlar los procesos intervinientes en la ejecución de dicha instrucción. El "reg. de direcciones de memoria" (RDM) que es usado para contener la dirección de la locación de memoria a la que se quiere acceder. El "reg. buffer de memoria o reg. auxiliar de memoria" (RBM) es la puerta de entrada/salida de la memoria en el se deposita la palabra de memoria leída o se toma la palabra a ser escrita en la memoria.

Un diseño de una máquina mínima sería:

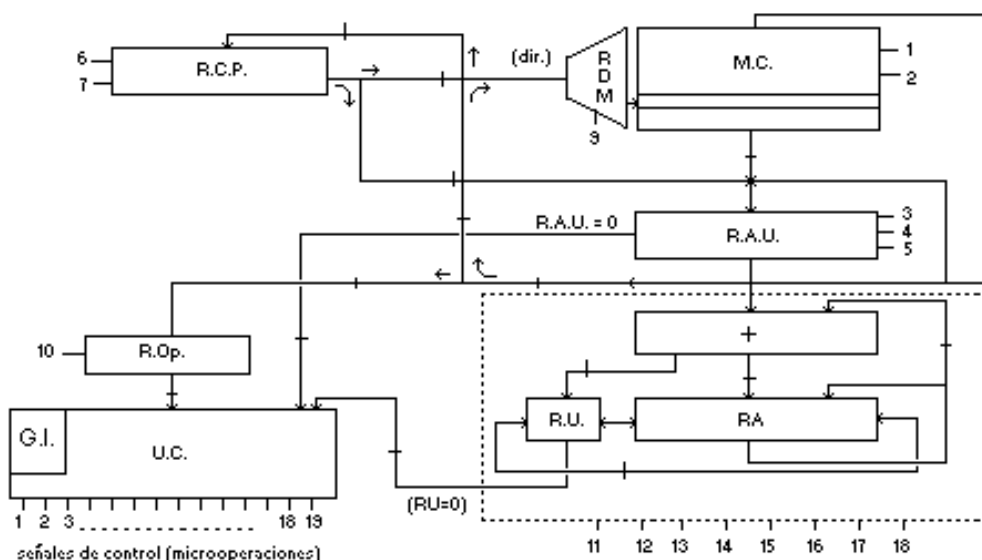


Figura 8-9.

Definición de los componentes.

Unidad de Control o Comandos (UC) es la encargada administrar el funcionamiento de la máquina. Con tal finalidad emite secuencialmente las señales de control o microoperaciones (MO) necesarias para cumplimentar los estados de búsqueda y ejecución. La UC posee terminales de salida; una por cada MO.

Téngase en cuenta además que el mismo número identifica el terminal de control del registro a través del cual al MO debe ingresar. Estos números están puestos solo a modo de ejemplo.

los impulsos que activan la UC y que sincronizan el funcionamiento general de la máquina, provienen de un generador de impulsos (GI).

La UC tiene registros asociados tal como el RCP y el RO.

Registro Contador de Programa (RCP), Es del tipo contador. Contiene la dirección de la próxima instrucción a ejecutarse. En el mismo se coloca, inicialmente, la dirección de comienzo del programa y luego es incrementado en 1 por cada instrucción que se ejecuta.

Registro Operación (RO), es donde se deposita el código de operación de la instrucción para ser analizado por la UC.

La **Memoria central (MC)** es ACTIVA, es decir, permite la lectura o escritura (aleatoria) de todos sus registros, pero además es BANALIZADA pues cualquiera sea la dirección de los mismos alojan por igual tanto instrucciones como operandos. La MC tiene asociados el RDM y el RAU.

Registro de Dirección de Memoria (RDM) es un registro del tipo acumulador y tiene por función la de alojar la dirección de la palabra cuyo contenido ha de leerse o escribirse.

Registro auxiliar (RAU) tiene la función de actuar en calidad de Registro Buffer de la Memoria Central (MC). En efecto, toda información (operando o instrucción) que egresa de la MC o que ingresa a la misma es transitoriamente alojada en el RAU. Además este registro puede incrementarse, siendo por lo tanto de tipo mixto, Acumulador y Contador.

Unidad Aritmética y lógica (UAL) es donde se realizan las distintas operaciones. Posee un registro denominado Acumulador (RA), un registro de un solo bit llamado Registro Unión (RU). El RA y RU se encuentran fuertemente acoplados, como si fuesen un solo registro al momento de ser operados. Y, por último un adicionador binario de n sumadores.

Funcionamiento de la máquina mínima.

La ejecución de toda instrucción requiere de dos etapas. Primero, localizar (leer) la instrucción en la memoria (Estado de búsqueda) y luego la ejecución propiamente dicha de la misma (Estado de ejecución).

Estado de búsqueda. En esta máquina la búsqueda de la instrucción se cumplimenta durante tres ciclos de reloj. En el primero se envía el RCP al RDM, en el segundo se realiza la lectura de la instrucción, conjuntamente se incrementa en 1 el RCP y en el tercero se envía el código de operación al ROP.

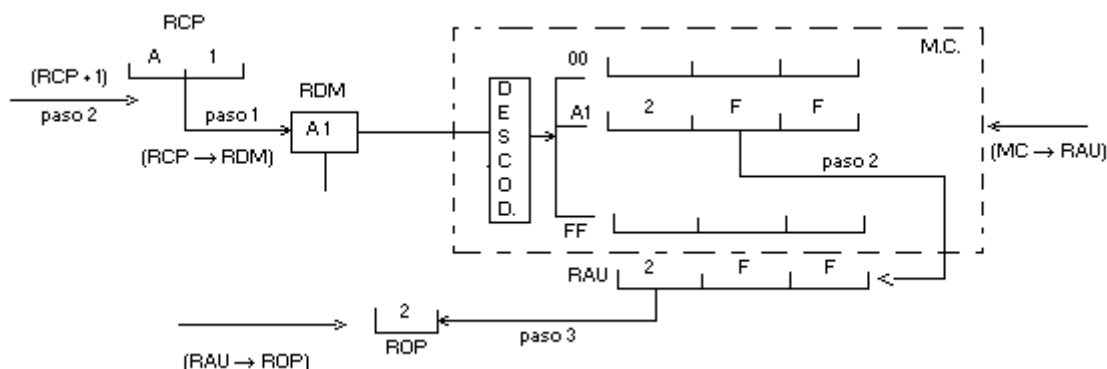


Figura 8-10.

En el esquema de la figura 8-10 podemos analizar el siguiente ejemplo: supongamos que al iniciarse la búsqueda de la instrucción el RCP contiene el valor A1h. Esto indica que la instrucción que debe buscarse en MC para ser ejecutada se encuentra en la palabra N° A1h, y si el contenido de la palabra A1h es el valor 2FFh, entonces se sucederán las siguientes operaciones:

En el ciclo de reloj 1: el valor A1h presente en el RCP es copiado en el RDM. De este modo, la palabra n° A1h se encuentra direccionada a través del decodificador de direcciones de la MC.

En el ciclo de reloj 2: se produce la lectura de la palabra direccionada. Por lo tanto, el contenido de la palabra A1h, es decir 2FFh, es copiado en RAU. Además este mismo ciclo de reloj se incrementa el contenido de RCP.

De este modo en el ejemplo presente, el RCP pasa a contener el A2h (A1h + 1h).

En el ciclo de reloj 3: el contenido del campo de operación de la instrucción presente en el RAU, es decir la cantidad 2h es copiada en el ROP. Así concluye el estado de búsqueda, debiendo retenerse los tres hechos importantes siguientes:

1. El código de operación de la instrucción se encuentra en ROP a disposición de la UC.
2. El RCP contiene la dirección de la palabra que sigue a la ocupada por la instrucción buscada.
3. El RAU contiene la instrucción buscada.

Estado de Ejecución. Durante este ciclo la UC proporciona las señales de control o MO necesarias para ejecutar la instrucción cuyo campo de operación se encuentra presente en el ROP. Los ciclos de reloj que se requieren para que se ejecute la instrucción dependen del número de MO que deben efectuarse secuencialmente.

Instrucciones de salto o bifurcación. Esta se lleva a cabo, reemplazando el contenido actual del RCP por la dirección a la que se quiere bifurcar o saltar. Es por este tipo de acciones (modifican el RCP), que el incremento del RCP para seguir con la secuencia lógica de un programa se realiza **antes** de ejecutar la instrucción (en el estado de búsqueda).

LENGUAJE MAQUINA Y ENSAMBLADOR.

En el nivel mas básico, el microprocesador responde a un listado de operaciones que se denomina programa maquina. Este esta escrito en código maquina siguiendo el formato de instrucciones que ya hemos visto y por supuesto en binario. En este formato los programas son muy difícil de comprender para los seres humanos y mucho mas difícil de generar.

Un primer paso para mejorar su interpretación, seria manejar los valores binarios en notación hexadecimal, pero de todas formas todavía el sistema es bastante complejo.

El lenguaje ensamblador utiliza palabras y frases para representar los códigos máquina del microprocesador y además palabras o etiquetas para referenciar posiciones de memoria.

El programa escrito de esta forma se denomina Simbólico o fuente y pasa a formato Objeto o Absoluto cuando es traducido a binario.

Lenguaje ensamblador
(programa simbólico o fuente)

Etiqueta	Nemotecnico	Operando	Comentarios
LONUM	LDY	\$70	; Carga Y con # de elementos en lista almacenada en la posición de memoria 0070h
	LDA	#\$FF	; Carga A con FFh (suponer número menor en FFh)
COMP	DEY		; Decrementa Y (registro Y es contador)
•	•	•	•
•	•	•	•

Figura 8-11.

La escritura de un programa en formato simbólico en lenguaje ensamblador, según se ve en la figura 8-11, normalmente tiene un formato restringido y podemos dividir cada sentencia en cuatro campos: (1) etiqueta, (2) nemotecnico, (3) operando, (4) comentarios. El campo etiqueta no se utiliza siempre, si colocamos algún nombre en este campo el mismo es asociado a la dirección de memoria correspondiente a esa posición del programa y el mismo nombre puede ser usado para referenciar esa dirección de memoria en cualquier punto del programa ya sea para localizar algún dato a operar o un sector del programa adonde bifurcar. El campo nemotecnico es el código simbólico exacto que proporciona el fabricante. Este campo indica al programa ensamblador la instrucción u operación que se desea realizar. El campo de operando contiene información sobre registros, datos o direcciones asociados con la operación. En este campo se puede referenciar directamente al operando o mencionando la etiqueta el mismo tenga asociada a alguna otra parte del programa. Por ultimo el campo comentario no es evaluado por el ensamblador y es usado solamente para comprender que ocurre en el programa.

Con los tres primeros campos el ensamblador genera el lenguaje correcto de maquina realizando básicamente la siguiente tarea: asocia cada etiqueta a la dirección correspondiente generando una tabla de referencia, reemplaza cada nemotecnico por su valor binario y cada operando por su dirección o valor binario, en caso de ser un nombre de etiqueta, la dirección correspondiente la toma de la tabla de referencia, asigna a cada operación-operando posiciones consecutivas de memoria.

Junto con el código Objeto, se proporciona un listado del programa simbólico y Objeto, listado de errores, referencias a las etiquetas utilizadas, etc.

Otra función del ensamblador es reservar espacio para los operandos, recordar el punto o dirección de arranque del programa, optimizar el programa agregando, modificando la secuencia de ejecución de las instrucciones, etc.. Algunos ejemplos se verán mas adelante en los ítems específicos. Algunas de estas funciones se pueden indicar mediante parámetros y/o pseudoinstrucciones (instrucciones al ensamblador).

Código máquina (objeto o absoluto)		Lenguaje ensamblador (programa simbólico o fuente)			
Dirección (hex)	Contenido (hex)	Etiqueta	Nemotecnico	Operando	Comentarios
0020	A4 70	LONUM	LDY	\$70	; Carga Y con # de elementos en lista almacenada en la posición de memoria 0070h
0022	A9 FF	COMP	LDA	#\$FF	; Carga A con FFh (suponer número menor en FFh)
0024	88		DEY		; Decrementa Y (registro Y es contador)
0025	08		PHP		; Introduce registro PS en pila (guarda señalizadores)
0026	D1 60		CMP	(\$60),Y	; Compara (A - M) pone C = 1 si $A \geq M$ pone C = 0 si $A < M$ (direccionamiento indirecto Y)
0028	90 2D		BCC	NOCHA	; Si C = 0, entonces bifurca a dirección simbólica NOCHA
002A	B1 60	NOCHA	LDA	(\$60),Y	; Carga A con nuevo numero mas bajo de la lista (usando direccionamiento indirecto Y)
002C	28		PLP		; Saca registro PS de la pila (restaura señalizadores)
002D	D0 24		BNE	COMP	; Si Z = 0, entonces bifurca a dirección simbólica COMP
002F	60		RTS		; Vuelta de subrutina

Figura 8-12

En la figura 8-12 se puede observar nuevamente el programa fuente junto con el programa objeto generado por el ensamblador, sin interesar la función de esta rutina y de las instrucciones (algunas de las cuales fueron modificadas con el fin de resaltar lo que seria el campo operación y el operando o dirección), se puede observar en la primer columna la posición hexadecimal donde esta cada palabra del programa y en la segunda columna el programa objeto que no es mas que una secuencia de valores hexadecimales (binario expresado en hexadecimal).

La programación en lenguaje ensamblador es un método de 'humanizar' las operaciones de un microprocesador; aunque sigue siendo bastante difícil su manejo, a este lenguaje lo denominamos de bajo nivel. Existen otros lenguajes como el BASIC, FORTRAM, PASCAL, etc. que facilitan aun mas la tarea del programador y no son tan rígidos en su formato de escritura. En estos lenguajes de alto nivel una sentencia de programa puede ser equivalente a 20 o 30 instrucciones de maquina.

MEJORANDO EL RENDIMIENTO.

Modos de direccionamiento. El modo de direccionamiento realizado en el ejemplo de maquina precedente es el denominado directo o absoluto, que queda definido a continuación junto a otras diferentes formas de realizarlo.

- *Modo directo:* La dirección de memoria del operando, está dada explícitamente como una parte de la instrucción.
- *Modo inmediato:* El operando está dado explícitamente en la instrucción.
- *Modo indirecto:* La dirección efectiva del operando está en el registro o en la locación de memoria, cuya dirección aparece en la instrucción.
- *Modo indexado:* La dirección efectiva del operador se obtiene sumando un valor "Índice" a la dirección dada en la instrucción.

transferencia entre registros. Ya hemos mencionado que normalmente se dispone no uno sino de varios registros de uso general para realizar las diferentes operaciones. Para permitir la transferencia entre ellos se conectan bus común; como se vio en la unidad 6; debiéndose activar las respectivas puertas de salida y entrada del registro emisor y receptor respectivamente.

secuenciamiento de las señales de control. La unidad de control deberá tener algún medio de generar la secuencia de señales de control. Se han usado una gran variedad de técnicas para resolver este problema, la mayoría de las cuales pertenecen a una de las siguientes categorías:

1. Control por Hardware.
2. Control Microprogramado.

Control por Hardware: Ya hemos visto en el ejemplo de máquina mínima cómo son las diferentes señales de control y cómo deben ejecutarse por ejemplo en diferentes ciclos de reloj (recordar el contador utilizado para generar señales visto en la máquina de programación fija). A su vez, cada acción puede generar nueva información (overflow, carry, cero, negativo, etc.). Estas señales denominadas Flags no son más que líneas de control que entran a la unidad de control, ésta posee un codificador/decodificador (circuito combinatorio) que genera las señales de control requeridas, dependiendo de los resultados parciales que se vayan dando.

Control microprogramado: Para describir este método, se comenzará definiendo a la palabra de control, cuyos bits individualmente representan a cada una de las señales de control. Por lo tanto cada uno de los pasos que forman la secuencia de control en la ejecución de una instrucción, define una única combinación de ceros y unos en la palabra de control.

Los microprogramas correspondientes al conjunto de instrucciones de máquina, están almacenados en una memoria especial, llamada memoria de microprograma. La unidad de control genera entonces la secuencia de señales de control para ejecutar una instrucción leyendo sucesivamente las palabras de control del correspondiente microprograma desde la memoria de microprogramas.

Como conclusión se resaltarán algunos puntos importantes relativos a las máquinas microprogramadas;

- 1.- El conjunto de instrucciones de una máquina, queda definido en su microprograma, por lo tanto es posible cambiar el conjunto de instrucciones de una CPU, simplemente alterando el contenido de la memoria de microprogramas. Esto ofrece una gran flexibilidad tanto al diseñador como al usuario de la máquina.
- 2.- Ya que el contenido de la memoria de microprograma es modificado muy infrecuentemente, usualmente se usan memorias de lectura solamente (ROM) para este propósito.
- 3.- Generalmente se provee en bus separado dedicado a la comunicación entre la CPU y la memoria de microprogramas.
- 4.- Ya que la ejecución de una instrucción de máquina implica varios accesos a la memoria de microinstrucciones, el tiempo de acceso de ésta juega un papel muy importante en la determinación del tiempo total de ejecución de cada instrucción.

EN BUSCA DE MAYOR VELOCIDAD.

Arquitecturas paralelo.

El procesamiento paralelo es una forma eficiente del procesamiento de la información que se basa en la explotación de sucesos concurrentes en el proceso de cómputos.

Concurrencia implica paralelismo, simultaneidad y pipelining. Sucesos paralelos pueden ocurrir en múltiples recursos durante el mismo intervalo de tiempo; sucesos simultáneos pueden ocurrir en el mismo instante de tiempo y sucesos pipeline pueden ocurrir en lapsos de tiempo solapados.

Estos sucesos concurrentes aparecen en un sistema de cómputos en varios niveles de procesamiento. El procesamiento paralelo demanda la ejecución concurrente de varios programas en el computador, en contraste con el procesamiento secuencial.

Hablaremos de niveles de paralelismo, a los cuales caracterizaremos de la siguiente manera, y detallaremos considerando de un mayor a menor nivel de paralelismo:

Multiprogramación, Multiprocesamiento: son acciones que se toman a nivel de programa o trabajo.

Tarea o Procedimientos: representan acciones que se toman dentro de un mismo programa, ejecutándose procesos independientes en forma simultánea.

Interinstrucciones: acciones a nivel de instrucción. Esto es, dentro de una misma tarea o proceso se pueden ejecutar instrucciones independientes en forma simultánea.

Intrainstrucciones: involucran acciones simultáneas que se pueden realizar para una misma instrucción. Por ejemplo, vectorización de operaciones escalares dentro de una instrucción compleja tipo DO, FOR.

Como puede notarse, el paralelismo de un mayor nivel se obtiene por medio de algoritmos, cuando en cambio los de menor nivel se logra con importante actividad de las componentes de hardware.

A los efectos de lograr un mayor grado de paralelismo, técnicas específicas del procesamiento distribuido son incorporadas a arquitecturas centralizadas. Algunas de las formas por las cuales puede obtenerse el paralelismo, son las siguientes:

Multicomputadoras: computadoras independientes, muy a menudo una de ellas actúa como supervisor, que realizan una tarea común en una sola ubicación.

Multiprocesadores: un conjunto de unidades de cómputo, donde cada una de las cuales tiene sus propios conjuntos de instrucciones y datos, pero comparten una misma memoria. Las computadoras multiprocesadores consisten en un número “n” (siendo n mayor o igual a dos) de procesadores que operan sobre una misma memoria y, se encuentran interconectados mediante canales que transmiten comandos de datos y control. Son controlados por un sistema operativo único.

Redes de Computadoras: básicamente involucra varias computadoras independientes conectadas mediante un canal de manera tal que los recursos disponibles en un punto de dicha red, puedan verse como disponibles para todos los integrantes en dicha red.

Procesador Pipeline: se trata de un único computador que puede realizar simultáneamente las operaciones de cálculos en determinadas secciones, con diferentes estadios de completitud. Los procesadores pipeline se basan en el principio de dividir los cálculos a realizar en una cantidad de unidades funcionales capaces de operar simultáneamente existiendo superposición.

Procesador array: es un grupo de unidades de cómputo donde cada una de las cuales realiza en forma simultánea la misma operación sobre diferentes conjuntos de datos. Estos procesadores operan sobre vectores. Las instrucciones del mismo son ejecutadas en serie, pero trabajan en forma paralela sobre vectores de datos.

Clasificación de FLYNN: Un procesador típico trabaja buscando instrucciones y operandos de la memoria principal, ejecutando las instrucciones y colocando los resultados en la memoria. Las distintas etapas asociadas con este procesamiento constituyen el ciclo de instrucción:

- Generar la dirección de la próxima instrucción.
- Buscar la instrucción.
- Decodificarla.
- Generar las direcciones de los operandos.
- Ejecutar la instrucción.
- Almacenar los resultados.

Las instrucciones pueden verse como una corriente o flujo de instrucciones que se desplazan de la memoria al procesador central, mientras que los operandos forman otra corriente o flujo, que en adelante llamaremos flujo de datos, desplazándose entre la memoria y el procesador central.

M.J.Flynn realizó una clasificación bastante útil del paralelismo presente en un procesador basado en el número del flujo de instrucciones y de datos simultáneos visto por el procesador durante la ejecución de un programa.

Teniendo en cuenta las combinaciones que se pueden presentar con estos dos ítems, flujo de datos y flujo de instrucciones, surgen estos cuatro grupos:

- **SISD** (Single Instruction - Single Data)
- **SIMD** (Single Instruction - Multiple Data)
- **MISD** (Multiple Instruction - Single Data)
- **MIMD** (Multiple Instruction - Multiple Data)

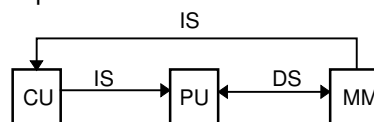
En la figura 8-13 vemos que esta categorización depende de la multiplicidad de sucesos simultáneos en los componentes del sistema.

Para comprender esta clasificación nos basta tener en cuenta tres componentes ya vistos:

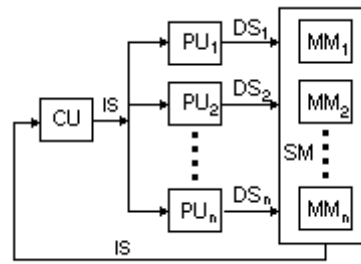
1. Módulos de memoria
2. Unidad de control.
3. Unidad de procesamiento o unidad aritmética y lógica.

El flujo de datos entre el módulo de memoria y el procesador circula en forma bidireccional, en categorías superiores vemos que se recurren a módulos de memoria múltiples que producen flujo de datos múltiples.

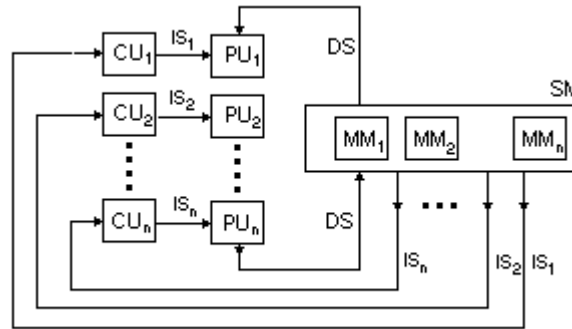
En el gráfico de la figura 8-13 podemos notar que cada flujo de instrucciones es generado por una unidad de control independiente. En el gráfico, no se indica el subsistema de entrada / salida ya que no es factor determinante en este tipo de clasificación.



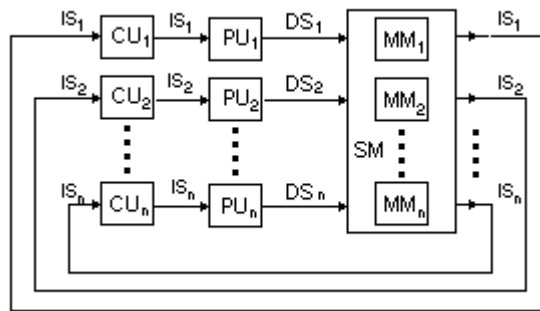
(a) SISD



(b) SIMD



(c) MISD



(d) MIMD

CU = Unidad de Control
PU = Unidad de proceso
MM = Módulo de memoria

IS = Flujo de Instrucciones
DS = Flujo de datos
SM = Memoria compartida

Figura 8-13.

SISD: Este tipo de arquitectura corresponde a la mayoría de las computadoras serie, disponibles actualmente, e indica (como puede observarse en la figura 8-13 (a)) una sola instrucción, un solo dato a la vez.

Las instrucciones se ejecutan secuencialmente pero pueden superponerse en sus diferentes etapas de ejecución (pipeline).

En muchos casos se recurre a varias unidades funcionales como ser de procesamiento y de memoria, pero siempre estarán supervisadas por una única unidad de control.

Como se vio anteriormente el proceso de ejecución de una instrucción implica generalmente cuatro etapas: búsqueda de la instrucción (IF) de la memoria principal; decodificación de la instrucción (ID); búsqueda del operando (OF), si es necesario en la ejecución; y la ejecución (EX) propiamente dicha.

En un computador con arquitectura clásica estas etapas deben completarse antes de ejecutar la próxima instrucción. Por ejemplo, si tendríamos en un programa un bloque de código con las siguientes características:

For i = 1 to 5
A(i) = A(i) * 2;

Las cinco veces que se pasa por la asignación, serán cinco instrucciones, con instrucciones completamente independientes de las otras.

SIMD: Si lográramos implementar el ejemplo anterior (dado en SISD) en forma paralela, en cinco unidades de procesamiento simultáneas bajo el control de una única unidad de control, ejecutaríamos las cinco instrucciones en forma simultánea, con lo cual se lograría ahorrar bastante tiempo. Esta idea la podemos visualizar en la figura 8-13 (b). El esquema es, tener una memoria, una unidad de control de la cual dependen varias unidades de ejecución que depositarán sus datos en memoria. Es decir, una sola

instrucción que ataca a muchos datos. Este esquema se conoce comúnmente como un procesador vectorial.

MISD: Esta arquitectura presenta n elementos procesadores; cada uno de ellos recibe distintos flujos de instrucciones que operan sobre el mismo flujo de datos. Es decir, se tienen múltiples instrucciones con un solo dato.

Los resultados de un procesador se convierten en la entrada (operandos) del próximo procesador y así sucesivamente. Esta arquitectura no ha recibido mucha atención hasta el momento y no existen implementaciones reales de la misma, lo cual significa que es solamente un esquema teórico.

MIMD: Esta categoría corresponde a la mayoría de los sistemas multiprocesos. En este esquema tenemos varias unidades de control, con varias unidades de ejecución. Es decir, se ejecutan muchas instrucciones con muchos datos.

En estos sistemas vemos que existirá una interacción entre los n procesadores ya que los flujos de datos se obtienen de la unidad de memoria compartida por todos los procesadores. Si los n flujos de datos se obtuvieran de subespacios disjuntos del módulo de memoria, sólo tendríamos múltiples SISD que no es otra cosa que un conjunto de n sistemas SISD (de un único procesador) independientes.

Teniendo en cuenta esa interacción entre los distintos procesadores se suele hacer una división entre los fuertemente acoplados y débilmente acoplados.

Paralelismo en sistemas de un solo procesador.

Cierto grado de paralelismo se puede conseguir en sistemas con un solo procesador. Ésto se consigue por hardware y software.

Los intentos realizados por hardware se basan en la multiplicidad de recursos y superposición temporal. En estos intentos es necesario realizar un balance de la velocidad de proceso de los distintos subsistemas con el objeto de evitar cuellos de botella y lograr así el incremento de la performance del sistema aumentando la cantidad de instrucciones ejecutadas en la unidad de tiempo.

Las técnicas empleadas a tal efecto son:

Multiplicidad de unidades funcionales: Consiste en aumentar la cantidad de unidades funcionales, correspondiente a la Unidad Aritmética y Lógica.

Paralelismo y Pipelining en la CPU: Aquí en lugar de aumentar la cantidad de unidades funcionales, se recurre a mejorarlas. El Pipelining consiste en la superposición de las distintas etapas de la ejecución de una instrucción.

Superposición de operaciones de CPU y de E/S: La idea es la ejecución simultánea del ciclo de CPU y la E/S. Básicamente se consigue con las técnicas de acceso directo a memoria, o con la utilización de procesadores dedicados exclusivamente a las operaciones de E/S.

Utilización de sistemas de memoria jerárquicos: El objetivo es disminuir la diferencia de velocidades entre CPU y el sistema de memoria, recurriéndose así a la cache que oficia de interface entre CPU y la memoria principal, etc.

Multiprogramación y Tiempo compartido: También se puede conseguir un mejor rendimiento del computador mediante opciones de software. Es sabido que tendremos programas de aplicación en que aparecerán grandes actividades de procesamiento, ocupando la mayor parte del tiempo la CPU, y otros por el contrario insumen mucho tiempo en operaciones de entrada/salida.

El objetivo es entonces de superponer la ejecución de estos programas balanceando el ancho de banda de las distintas unidades funcionales del procesador, obteniéndose de esta manera la superposición de operaciones de entrada/salida y de CPU.

El problema que puede aparecer con este método es que si existen procesos que demanden una utilización de CPU demasiado prolongada, no darían lugar a la ejecución de otros procesos, apareciendo tiempos de espera demasiado largos y por lo tanto perdiéndose las bondades de la multiprogramación. El método de tiempo compartido (Time Sharing) resuelve este problema asignando periodos limitados de tiempo a cada uno de los procesos.

Arquitectura PIPE-LINE.

Ya hemos mencionado que para lograr operaciones concurrentes en un procesador se utilizan dos técnicas básicas: paralelismo y pipelining.

El paralelismo conseguía la concurrencia multiplicando las componentes del hardware tantas veces como sea posible, de modo que las diferentes etapas del proceso se ejecuten simultáneamente.

Pipelining consiste en desdoblarse la función a realizarse en varias partes, es decir, divide la tarea en una secuencia de sub tareas cada una de las cuales se ejecuta en una etapa de hardware especializada que trabaja concurrentemente con otra de las etapas del pipeline. Así como el agua fluye a través de una tubería (pipeline) las instrucciones o datos fluyen a través de las etapas de un computador digital pipeline a una velocidad que es independiente de la longitud de la tubería (número de etapas del pipeline) y depende solamente de la velocidad a la cual los datos e instrucciones pueden ingresar al pipeline.

Esta velocidad a su vez depende del tiempo que tarde el dato en atravesar una etapa. Este tiempo puede ser significativo ya que el computador no sólo desplaza los datos o instrucciones de etapa en etapa sino que en cada una de ellas se realiza alguna operación sobre los mismos. Como ejemplo en el caso de las instrucciones tendremos operaciones de búsqueda, decodificación y ejecución.

Pipeline Físico o de Instrucción: Veamos como primer ejemplo de un pipeline (de cuatro etapas) el proceso de ejecución de una instrucción en un computador digital que como vimos, modela 4 pasos principales: levantar la instrucción de memoria (Instruction Fetch - IF); identificar la operación que debe efectuarse (Instruction Decoding - ID); levantar los operandos de ser necesarios en la ejecución (Operand Fetch - OF); y por último ejecutar la operación aritmético lógica que ha sido decodificada. Debe entenderse que antes de comenzar la ejecución de una nueva instrucción, debe completarse los cuatro pasos antes indicados.

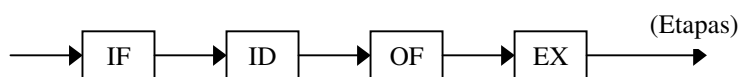


Figura 8-14 (a) Un procesador pipeline

En la figura 8-14 (b) podemos ver la ejecución superpuesta de las instrucciones sucesivas, que se comentaba con anterioridad. Luego de haber cargado el primer pipe un procesador pipeline produce un resultado de salida por cada ciclo (donde ciclo de pipeline se considera aproximadamente igual al tiempo que tarda la etapa más lenta).

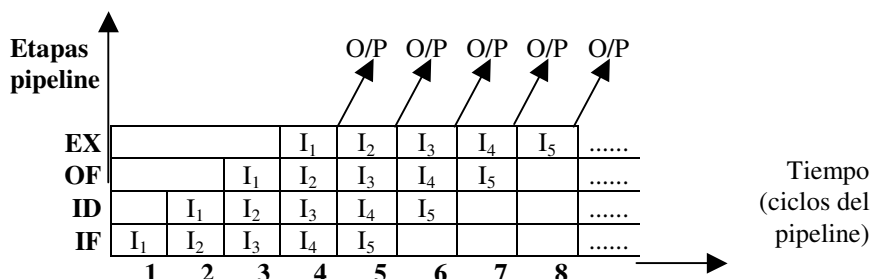


Figura 8-14 (b) Diagrama espacio-temporal para un procesador pipeline.

Esto muestra que el ciclo de una instrucción se reduce a un cuarto del tiempo de ejecución original en relación a un procesador sin pipeline como lo muestra la siguiente figura:

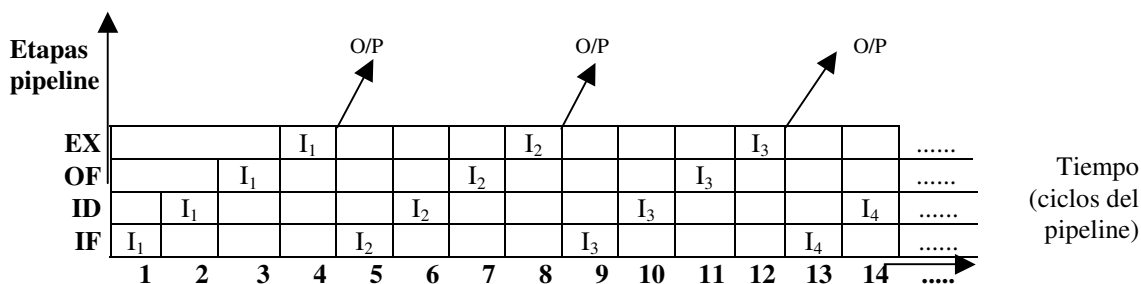


Figura 8-14 (c) Diagrama espacio-temporal para un procesador no-pipeline.

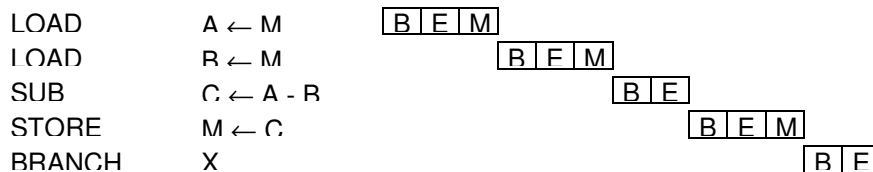
Veamos algunos ejemplos más de pipelines pero en este caso de dos y tres etapas. Algunas instrucciones requieren dos etapas para ser ejecutadas como la mayoría de las operaciones entre registros:

- **B**: Búsqueda de la instrucción.
- **E**: Hace una simple operación aritmética (ADD/SUB), lógica o de desplazamiento.

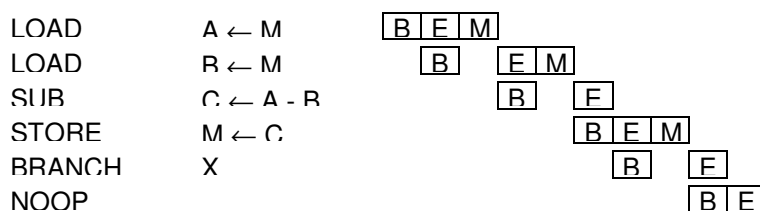
y entre instrucciones que requieren tres etapas como load y store:

- **B**: Búsqueda de la instrucción
- **E**: Calcula la dirección necesaria para hacer un acceso a memoria.
- **M**: Hace la operación de memoria a registro o de registro a memoria.

A continuación se muestra la ejecución de una secuencia de instrucciones en ausencia de un pipeline, lo cual sin ninguna duda y adelantándonos a los próximos ejemplos constituye un claro desperdicio.

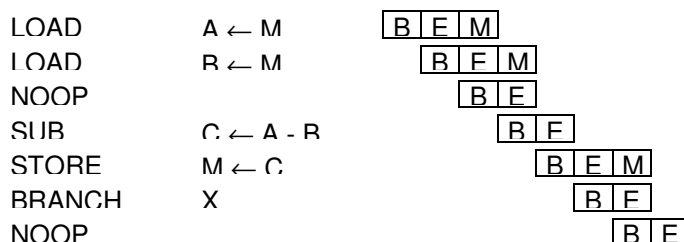


Veamos ahora esta misma secuencia de instrucciones ejecutada en un pipeline de dos etapas.



Con este esquema, las fases B y E de dos instrucciones diferentes pueden ejecutarse simultáneamente. Si suponemos que la memoria tiene una sola puerta de acceso, como es el caso de RISC I y RISC II por ejemplo (ver Arquitecturas CISC y RISC), solamente un acceso a memoria puede estar llevándose a cabo en un instante dado. Esto implica que cuando la memoria de datos está siendo accedida por una instrucción de LOAD o STORE, el resto del pipeline está suspendido, pues un acceso de búsqueda no puede ser procesado al mismo tiempo. Esto requiere la introducción de un estado de espera en algunas instrucciones. Debido a la interrupción de la secuencia normal de instrucciones producida por el branch, se introduce un NOOP (no operation) para simplificar el diseño.

Una manera de que la CPU vea como si tuviese dos puertas de memoria, es incorporar una memoria cache de instrucciones. Efectivamente esta memoria cache aparece como una puerta independiente para la CPU, siempre que no ocurra un *miss* (faltante). La CPU verá entonces dos puertas separadas de memoria, una para instrucciones y una para datos. El esquema quedaría:



Al quitar la limitación impuesta por una única puerta de acceso a memoria, el ciclo de acceso al dato de una instrucción LOAD o STORE (**M**) puede ocurrir en paralelo con el ciclo de cálculo (**E**) de la próxima instrucción. Esto no ocasiona ningún inconveniente de dependencia de datos para las instrucciones STORE, pero si en cambio para las instrucciones LOAD. La etapa de cálculo de la instrucción que sigue al LOAD no debe depender del valor que está siendo cargado. Esta situación puede ser examinada o bien por el compilador, quien en tal caso inserta un NOOP, o mediante extra hardware, suspendiendo el pipeline mientras espera que el dato llegue de memoria.

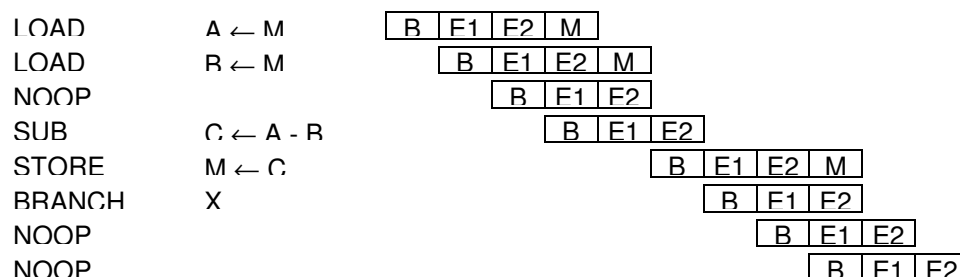
En el caso más favorable la velocidad de ejecución puede ser hasta tres veces la del sistema sin pipeline.

Hasta aquí hemos considerado que las tres etapas son aproximadamente de igual duración.

Debido a que la fase **E** generalmente involucra una operación de ALU, su ejecución es mayor que las demás, pudiendo ser entonces dividida en dos etapas:

- **E1**: Lectura del registro.
- **E2**: Operación de ALU y escritura de un registro.

Lo cual nos conduce a un Pipeline de cuatro etapas como vemos en el siguiente esquema:



Hasta cuatro instrucciones pueden estar ejecutándose a la vez, por lo tanto la velocidad puede ser, en el caso más favorable, de hasta cuatro veces más que la de un sistema sin pipeline.

Pipeline Digital o Lineal:

Mientras un ítem en particular fluye a través del pipeline, éste ocupa sólo una etapa por vez. A medida que transcurre el tiempo la etapa que quedo vacante por un ítem, será ocupada por el siguiente. Este uso concurrente de diferentes etapas por diferentes ítems se lo conoce por solapamiento. El resultado neto es que la máxima velocidad a la cual los nuevos ítems pueden ingresar al pipeline depende exclusivamente del tiempo máximo requerido para atravesar una etapa y no del número de éstas.

La utilización del pipeline se traduce en una mejora del comportamiento o performance del procesador. Ésto es, si una función se lleva a cabo en T segundos en un procesador de diseño convencional, al utilizar un procesador pipeline de N etapas esa misma función podrá realizarse en T/N segundos. Ésto es a nivel teórico, dado que, la mejora de performance dependerá de la cantidad de operaciones que puedan estructurarse en pipeline y también de calidad de la partición o subdivisión de cada una de esas operaciones. La estructuración en pipeline de un procesador se realiza en diferentes niveles como aritmética de punto flotante, ciclo de instrucción, procesador de entrada/salida, etc..

Esta división o partición de una función en varias subfunciones se podrá realizar si se dan las siguientes condiciones:

1. La evaluación de la función básica es equivalente a la evaluación secuencial de las subfunciones que la constituyen.
2. Las entradas a una subfunción provienen únicamente de las salidas de subfunciones previas en la secuencia de evaluación.
3. Excepto el intercambio de entradas y salidas no existe otra vinculación entre las subfunciones.
4. Debe disponerse del hardware necesario para la evaluación de dichas subfunciones.
5. El tiempo requerido por estas unidades de hardware para evaluar las subfunciones es aproximadamente el mismo.

Este hardware mencionado en el punto 4 es el correspondiente o asociado con las distintas etapas del pipeline.

A diferencia del pipeline físico (tubería), el digital no maneja los datos en forma continua. Éste maneja los datos en forma interrumpida y los va desplazando de etapa en etapa sincronizando con un reloj.

Dado que la lógica que realiza el procesamiento en cada etapa generalmente carece de memoria y dado el carácter discontinuo de la aparición de los datos en las etapas se impone la utilización de un elemento de memoria en la entrada y salida de cada etapa. De esta manera, bajo el control del reloj asociado a cada elemento de memoria, evitamos que los datos ingresen a una etapa antes que haya finalizado el proceso anterior.

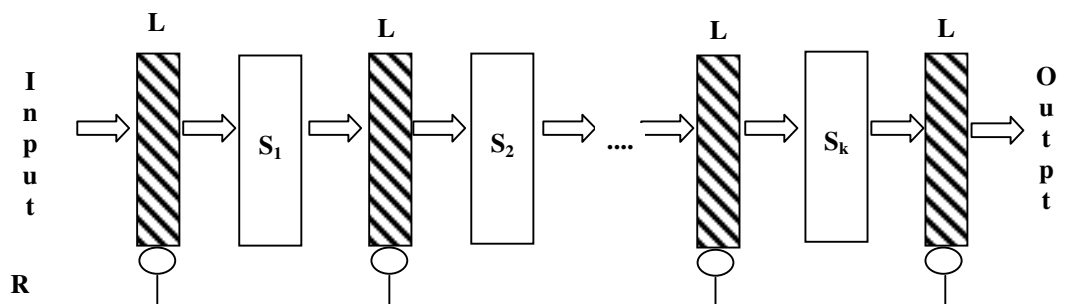


Figura 8-14 (c) Estructura de un procesador pipeline digital

L: Latch
 S_i: etapa i-ésima
 R: reloj

En la figura 8-14 (c) se puede ver la estructura básica de un pipeline digital o lineal. Éste consiste de una cascada de etapas de procesamiento. Dichas etapas son circuitos que efectúan operaciones aritméticas o lógicas sobre los datos que fluyen a través del pipe., y se encuentran separadas por registros de muy alta velocidad encargados de almacenar los resultados intermedios entre etapas, llamados latches.

Pipeline vs. Solapamiento.

Los conceptos de pipelining y solapamiento tienen significados parecidos pero no son equivalentes, pese a que en algunos casos se los utiliza indistintamente. Ambos se basan en la idea de partición o subdivisión de una función, pero se aplican en distintos entornos.

Se dice que podrá existir pipeline cuando se den las siguientes condiciones:

1. Cada evaluación de la función básica es independiente de las anteriores.
2. Cada evaluación requieren aproximadamente la misma secuencia de subfunciones.
3. Cada subfunción se encadena perfectamente con la anterior.
4. Los tiempos utilizados para evaluar las distintas subfunciones son aproximadamente iguales.

Solapamiento se podrá utilizar cuando se den las siguientes condiciones:

1. Existe dependencia entre las distintas evaluaciones.
2. Cada evaluación puede requerir una secuencia diferente de subfunciones.
3. Cada subfunción tiene un propósito distinto.
4. El tiempo por etapa no es necesariamente constante, sino que depende de la etapa y de los datos que la atraviesan.

En muchos casos el pipeline propiamente dicho y que detallamos en primer lugar, se lo suele llamar sincrónico o estático, por otra parte el solapamiento se lo suele llamar pipeline asincrónico o dinámico.

Un ejemplo de pipeline estático es el de una unidad destinada a realizar sumas en punto flotante sobre datos provenientes de otra unidad. Aquí la función básica, *suma en punto flotante* es relativamente simple, el resultado de una suma depende del anterior, próximo o de cualquier otro. Además una vez que se realizó la partición, todas las sumas siguen la misma secuencia.

El ejemplo más simple de solapamiento o pipeline dinámico se da en un procesador donde la entrada/salida es manejada por un procesador, los cálculos por otro y la comunicación se realiza a través de un módulo de memoria común.

La realización de una función en este procesador alternará entre el procesador de cómputo y el de entrada/salida.

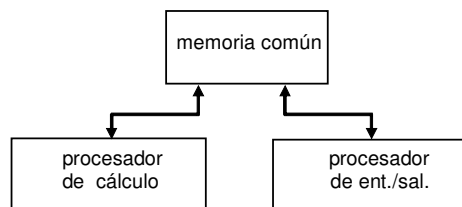


Figura 8-15

En otros casos la evaluación de una función sólo requerirá de un procesador. En este caso vemos que las distintas operaciones están menos sincronizadas que en el caso del sumador en punto flotante. La participación de la función básica varía dinámicamente y aún el tiempo empleado en cada etapa es difícil de predecir.

Otro ejemplo de operación por solapamiento es el de ejecución de instrucciones de CPU's de alta velocidad. En este caso el hardware podrá estar dividido en un número fijo de etapas como en el caso del sumador de punto flotante, pero de acuerdo con las diferencias de formato de las instrucciones cada una puede desarrollar un camino diferente a través del pipeline.

PROBLEMAS DE LOS PIPELINES

Dependencia entre instrucciones y riesgos.

En una máquina secuencial clásica todas las operaciones que involucran la ejecución de una instrucción se completan antes que comience la próxima. En el caso de las máquinas SISD podemos intentar realizar pipeline a nivel de ejecución de instrucciones.

Esto consistirá en solapar la ejecución de las mismas de manera que algunas de las operaciones necesarias para las instrucciones $i+1$, $i+2$, ..., puedan realizarse antes que finalice la ejecución de la instrucción i . Puede ocurrir que las operaciones realizadas por $i+1$, $i+2$, dependan de los resultados de i que aún no fue completada. Esta dependencia entre instrucciones crea riesgos que deben evitarse y resolverse.

Se reconocen tres clases de riesgos, read-after-write, write-after-read y write-after-write. Veamos las diferencias considerando este segmento de código de programa:

•
•
Almacenar en x
•
Sumar x
Almacenar en x
•
•
Almacenar en x
•
•

El riesgo read-after-write entre dos instrucciones i y j (siendo j posterior a i) se presentara cuando j trata de leer un operando que es modificado por i . Si la operación en i que modifica el operando no se completa antes que j lo acceda, entonces j leerá el valor erróneo. En el ejemplo el riesgo existirá entre el primer *almacenar en x* y el *sumar x*, si el sumar x extrae el operando antes que haya sido actualizado por almacenar.

El write-after-read ocurre cuando la instrucción j desea modificar un operando que es leído por i . Si j modifica el operando antes que i lo haya leído, este dato así leído será erróneo. Este riesgo puede existir entre el sumar y el segundo almacenar en x .

Finalmente el write-after-write existe cuando i y j tratan de actualizar el mismo operando, en el caso que j lo haga en primer lugar. En nuestro ejemplo se el segundo almacenar se realiza después del tercero, obviamente el valor en x no es el que el programa espera.

Manejos de saltos.

Uno de los problemas que ocasiona el uso de pipeline se da con la instrucción de bifurcación o salto. Recordemos que esta instrucción se ejecutaba colocando la dirección a la que debía ir en el Registro contador de programa. Ahora, si nos fijamos en el ejemplo siguiente veremos que cuando se ejecuta el salto, la próxima instrucción ya fue leída y por lo tanto se ejecutara erróneamente. Estos problemas ocasionados por la aparición de saltos en el programa pueden ser obviados mediante la alteración en el flujo normal de ejecución del programa. Tal técnica se la denomina usualmente como **salto retardado (delayed branch)**. La siguiente tabla muestra en la primera columna las instrucciones de un programa que incluye entre las mismas a un JUMP. En la segunda columna introducimos un NOOP después del JUMP en función de regularizar el pipeline. Si ahora alteramos el orden de ejecución de las instrucciones, como se muestra en la tercer columna, ejecutándose primero el JUMP y posteriormente el ADD, conseguimos una mejora en el rendimiento.

Dirección	Salto normal		Salto retardado		Salto retardado (optimizado)	
300	LOAD	X . A	LOAD	X . A	LOAD	X . A
301	ADD	1 . A	ADD	1 . A	JUMP	305
302	JUMP	305	JUMP	306	ADD	1 . A
303	ADD	A . B	NOOP		ADD	A . B
304	SUB	C . B	ADD	A . B	SUB	C . B
305	STORE	A . Z	SUB	C . B	STORE	A . Z
306			STORE	A . Z		

En las siguientes figuras podemos apreciar la diferencia en la ejecución:

300	LOAD	X . A	B	E	M
301	ADD	1 . A	B	E	
302	JUMP	306	B	E	
303	ADD	A, B	B	E	
306	STORE	A . Z	B	E	M

La instrucción 303 se ejecuta en forma errónea ya que fue leída antes de que se modifique el RCP.

300	LOAD	X . A	B	E	M
301	ADD	1 . A	B	E	
302	JUMP	306	B	E	
303	NOOP		B	E	
306	STORE	A . Z	B	E	M

(a) Primer solución: Insertando NOOP

300	LOAD	X . A	B	E	M
301	JUMP	305	B	E	
302	ADD	1 . A	B	E	
305	STORE	A . Z	B	E	M

(b) Solución mejorada: Intercambiando el orden de las instrucciones

Como la instrucción de ADD se busca antes de que la instrucción JUMP tenga oportunidad de alterar el contador de programa, el programa mantiene la semántica original.

Este procedimiento aplicado aquí a un salto incondicional, no puede aplicarse siempre en casos de saltos condicionales, ya que la condición que está examinando el branch puede ser alterada por la instrucción que la precede. En estos casos el compilador no debe hacer el intercambio de instrucciones. Diversos estudios llevados a cabo, muestran que la mayoría de los saltos condicionados pueden ser optimizados de esta forma.

Con las instrucciones LOAD, se procede de una forma similar. El registro sobre el cual se va a efectuar el LOAD es bloqueado por la CPU. La secuencia de instrucciones se continúa ejecutando hasta que alguna instrucción requiera el registro bloqueado, en este momento la CPU debe esperar la finalización del LOAD. Una mayor eficiencia se logra si el compilador puede reacomodar instrucciones mientras el LOAD está en el pipeline.

INTERFACES CON PERIFÉRICOS.

Los sistemas de cómputo se comunican con el "mundo exterior" a través de periféricos, tales como unidades de vídeo, impresoras, unidades de disco y de cinta, lectores de tarjetas, convertidores analógicos-digitales, etc. La comunicación entre el microprocesador del sistema y los periféricos se efectúa a través de unidades especiales denominadas interfaces. En la Figura 16, se muestra un diagrama en bloques de un sistema digital compuesto por un microprocesador (μP) comunicado con 4 periféricos a través de 2 interfaces. Los bloques asociados a los periféricos incluyen su unidad controladora propia. El μP se comunica con el periférico 1 a través de la interface 1; y con los periféricos 2, 3, 4 mediante la interface 2. Durante las operaciones de entrada/salida (E/S), cada interface tiene asignadas diversas funciones: convertir la información de acuerdo a los formatos utilizados por el μP y el periférico; direccionar la información hacia o desde el periférico de interés; informar sobre el estado del periférico (disponible u ocupado); reconciliar diferencias de tiempo entre el μP y el periférico (sincronización); etc.

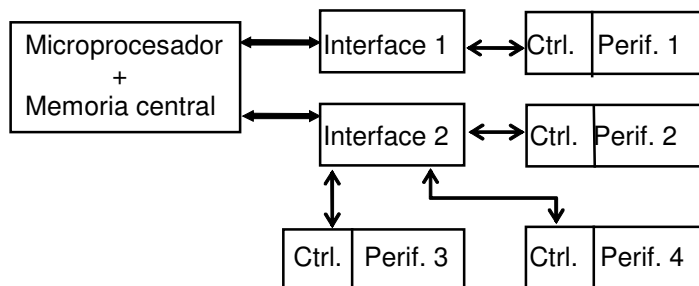


Figura 16: Diagrama de bloques de un sistema digital comunicado con periféricos.

La simultaneidad de operaciones.

El funcionamiento del sistema del sistema de la Figura 16 involucra tanto la ejecución de un programa en el μP (programa principal), como la ejecución de transferencias de información con las unidades periféricas (operaciones de E/S). La mayor velocidad de operación del sistema se logra cuando ambos tipos de ejecuciones se realizan con el mayor grado de simultaneidad. Existen al menos 4 modos para la ejecución de operaciones de E/S:

- *Modo bloqueado:* se interrumpe el programa principal para ejecutar operaciones de E/S, durante las cuales el μP transfiere información al "buffer" del controlador del periférico, y espera inactivo (bloqueado) que el periférico se desocupe para efectuar una nueva transferencia.
- *Modo por prueba de estado:* en el programa principal se intercalan sentencias que permitan reconocer el estado del periférico (disponible u ocupado). Si está disponible, el μP ejecuta una transferencia de E/S; si está ocupado, el μP continúa con la ejecución del programa principal. La mayor dificultad consiste en la complejidad que adopta el programa principal, tornando tediosa la tarea del programador.
- *Modo por interrupción de programa:* en lugar de controlar el estado de los periféricos mediante el programa principal, son los mismos periféricos quienes "avisan" al μP , el que permite suspender el

programa en ejecución para activar el programa de E/S y efectuar una nueva transferencia de información. Finalizada la transferencia, el μP continúa con la ejecución del programa principal.

- **Modo Canal:** la transferencia de información se efectúa directamente entre la memoria central y periférico durante un solo ciclo de reloj, y sin intervención del μP . La transferencia está a cargo de un unidad automática de intercambio denominada canal (en un diagrama como el de la figura 16, el canal es parte de la interface). Si el bloque de memoria a transferir no está siendo utilizado por el programa, la ejecución del mismo no se suspende. En caso contrario, el programa se suspende sólo durante el ciclo que dura la transferencia. Así cuando se necesita efectuar una transferencia, el canal "roba" sólo un ciclo de reloj al μP . De esta manera, la simultaneidad entre las ejecuciones del programa principal y de las operaciones de E/S es casi total.

Transferencias de Información.

Los intercambios de información entre la memoria central y los periféricos suelen clasificarse en dos categorías generales: a) las transferencias programadas; y b) las transferencias por acceso directo a memoria (por robo de ciclo).

Transferencias programadas.

Las transferencias programadas se efectúan siempre bajo control de un programa ejecutado por el μP . En la figura 17 se muestra un diagrama en bloques típico de sistema de cómputos con transferencia de información programada. El μP envía la instrucción de E/S al decodificador a través del bus de control. Para efectuar una operación de E/S, el decodificador selecciona el canal adecuado del multiplexor o del demultiplexor (según se trate de una entrada o salida, respectivamente), para contactar el periférico deseado. Luego, el μP recibe el estado del periférico seleccionado por medio del registro de estado de los periféricos, determinando si éste se encuentra libre u ocupado. Si el periférico está libre, se transfiere la información a través del bus de datos.

En el caso específico en que se efectúe una operación de salida, la información se transfiere al periférico a través de un grupo de biestables (Flip-Flop) del canal demultiplexor, en donde se la retiene hasta completar la transferencia. De esta forma, se elimina cualquier error que pudiera ocurrir debido a que la mayoría de los periféricos son más lentos que los μP .

Las transferencias programadas pueden ser clasificadas en tres tipos: a) sincrónicas, b) asincrónicas; y c) por interrupción de programa. Aunque no nos extenderemos en los temas, daremos algunas nociones básicas sobre estos tipos de transferencias.

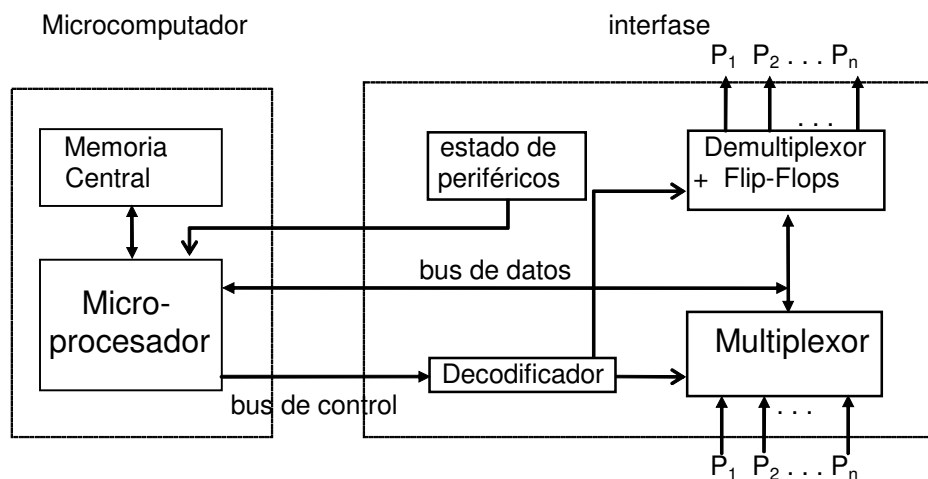


Figura 17: Diagrama en bloques de un sistema digital con transferencia de información programada.

Las transferencias sincrónicas son en general, las que requieren implementar la menor cantidad de "software" y de "hardware". En este tipo de transferencia, el periférico debe responder a la operación de E/S, dentro de ciertos tiempos especificados por el μP .

Las transferencias asincrónicas (técnicamente denominadas "handshaking") son comúnmente usadas en sistemas de microcómputo. para realizar una transferencia asincrónica, el μP efectúa los siguientes pasos: 1) testea el estado del periférico; 2) activa el periférico cuando éste se encuentra libre; 3) efectúa la transferencia de datos ("input" o "output"); y 4) desactiva el periférico.

Las transferencias por interrupción de programa son particularmente útiles cuando se dispone de periféricos lentos, o en aplicaciones donde se desconoce la probabilidad de ocurrencia de una transferencia (por ej., en redes de comunicación). En las transferencias por interrupción de programa, la iniciativa la tiene el μP que cumple los siguientes pasos: a) reconoce la presencia de la interrupción al final de una instrucción del programa principal, b) almacena el contador de programa, c) ejecuta una

rutina para procesar la interrupción; d) almacena sus registros internos (de trabajo y de estado), e) ejecuta la transferencia de datos bajo control por "software", y f) recupera su estado previo a la interrupción para continuar con la ejecución del programa principal.

Transferencias por acceso directo a memoria.

las transferencias por acceso directo a memoria (ADM) son controladas totalmente por "hardware"; y, por tal motivo, las interfaces son más complejas que las requeridas por las transferencias programadas. En la Figura 18 se muestra un diagrama en bloques típico de un sistema de cómputos con transferencia de información por ADM.

Cuando el programa principal requiere efectuar una operación de E/S, contacta un periférico, el cual pone en 1 la bandera de ADM. Entonces, la lógica de ADM solicita al μP ingresar en un estado "ocioso" (típicamente, mediante una entrada del tipo WAIT o HOLD en Req). Cuando el μP ingresa en el estado "ocioso", se lo notifica a la lógica ADM mediante la señal Ack. En tal situación, los buses de datos y de dirección se aíslan de μP mediante puertas de tercer estado. Esto permite que una lógica externa tome el comando de esos buses para ejecutar la transferencia de información por ADM. La señal de control se suministra a la lógica de ADM para indicar que se efectuará una operación de salida o de entrada en la memoria (R/W). El registro de dirección se carga con la dirección inicial del bloque de datos a transferir y el intercambio de datos entre la memoria central y el periférico se ejecuta a través del bus y del registro de datos. A continuación de cada transferencia, se incrementa el registro de dirección mediante la señal Inc. Cuando se concluye el intercambio de información, el periférico envía una señal de control que efectúa el "reset" de la bandera de ADM y de la señal Req. Esta última acción permite al V continuar con su tarea normal.

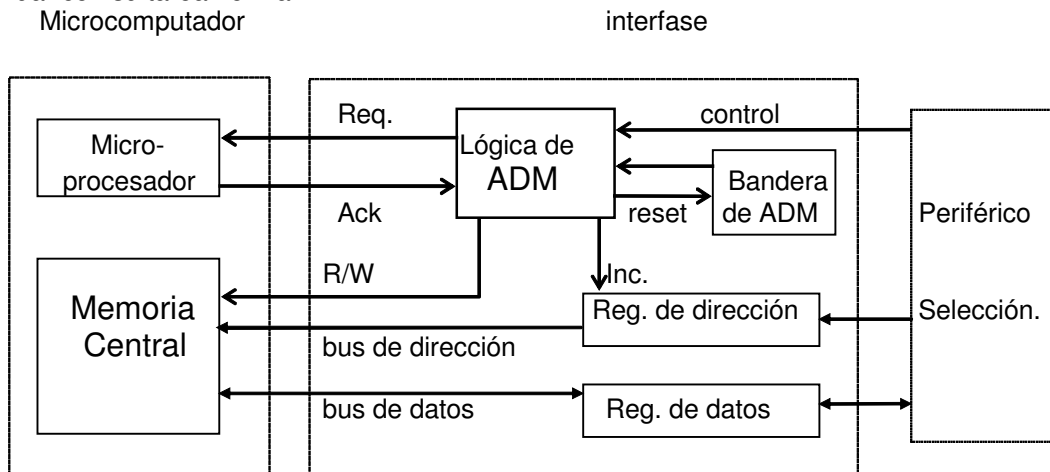


Figura 18: Diagrama en bloques de un sistema digital con transferencia de información por ADM.

ARQUITECTURAS CISC Y RISC.

Soportado en gran medida por el avance de la tecnología de circuitos integrados la tendencia en computadoras fue aumentar la complejidad de su arquitectura.

Con la idea de que al contar con un set más poderoso simplificaría los compiladores y además, que al crear instrucciones de máquina cada vez más parecidas a la sentencia del lenguaje de alto nivel se rompería el cerco entre lenguajes assembler (bajo nivel) y el lenguaje de programación, consiguiendo una mayor velocidad de proceso, se crearon computadoras de uso general con instrucciones cada vez mas complejas. A este grupo de maquinas las podemos agrupar como del tipo CISC (computadoras con set de instrucciones complejo).

Pero investigaciones realizadas sobre esta tendencia y sobre las implementaciones que generaron, dieron resultados negativos, como ser, un elevado tiempo de diseño, aumento de la probabilidad de errores y en algunos casos implementaciones inconsistentes.

Aún más, los compiladores al realizar su tarea no recurrían a las instrucciones más complejas, que un poco era la razón de estas nuevas estructuras.

Además debe tenerse en cuenta que esas instrucciones complejas eran muchas veces responsables de retardos en la performance de todo el sistema.

Sumado a ésto que la unidad de control para poder soportarlas debe ser más compleja que lo usual y, que al ser microprogramada esa complejidad adicional provoca mayor tiempo en su propia ejecución.

Con el propósito de explorar alternativas que subsanen esos inconvenientes, surgen las maquinas RISC (computadoras con set de instrucciones reducido).

La hipótesis es que al contar con un set reducido de instrucciones, se puede diseñar una arquitectura en VLSI (integración en muy grande escala) que haga uso de los recursos en una forma más eficiente que en CISC.

También se debe lograr una reducción en el tiempo de diseño, en los errores de diseño y en el tiempo de ejecución de las instrucciones.

Descripción de las características del RISC.

En desarrollos llevados a cabo por distintos grupos, es común observar que a veces usan una misma terminología o vocablo para designar cosas distintas, o que designan lo mismo con terminologías diferentes. RISC no es una excepción, o dicho en otras palabras, no todos definen RISC de la misma manera, quizá con el afán de poder decir que la máquina en cuestión es RISC.

Diferentes investigadores han propuesto las características que debe cumplir un computador para ser RISC. Como conclusión a esas propuestas y a la luz de las arquitecturas RISC más recientes, no se puede definir una grilla de características que debe cumplir una arquitectura para ser RISC, es común encontrar que los diseños más recientes carecen de alguno de los 'dogmas' establecidos, sino más bien hablar de las características típicas que podemos encontrar en las arquitecturas RISC. Éstas son:

- Operaciones de ciclo único.
- Diseño Load/Store, Registro a Registro.
- Control por hardware.
- Reducido número de instrucciones y modos de direccionamiento.
- Formato fijo de instrucciones.
- Pipelining.
- Maximización de uso de Registros.

Operaciones de ciclo único.

RISC I, establecía que las instrucciones deberían ser tan rápidas y no más complicadas que las microinstrucciones de las máquinas corrientes tales como PDP11 o VAX. Ejecutar una instrucción compleja que necesita varios ciclos para ejecutarse requiere convencionalmente una combinación de cierta lógica y microcódigo. Normalmente implementar esta instrucción compleja significará una ejecución más rápida que si la misma fuese implementada como una secuencia de instrucciones primitivas. Ejemplos de tales instrucciones son la multiplicación y división en punto fijo y las operaciones en punto flotante. La pregunta a hacerse es entonces: ¿cuándo se justifica implementar una instrucción compleja del tipo mencionado?. La respuesta es simple: se justificara siempre que la frecuencia con que se usen equilibre el costo extra asociado con su implementación, y más importante que esto es que, estas instrucciones no disminuyan la velocidad de ejecución de las instrucciones primitivas. La mayoría de los estudios sobre la frecuencia con que aparecen determinadas instrucciones mostraron un alto uso de instrucciones primitivas como LOAD, STORE, BRANCH, COMPARE, ADD. Si la presencia de un set de instrucciones más complicado significa agregar un nivel de compuertas extra a un ciclo básico de máquina de 10 niveles, la velocidad de la CPU ha sufrido una degradación de un 10%. La frecuencia de uso y el mejoramiento del rendimiento de funciones complejas debe primero superar esta degradación del 10 % y luego superar el costo adicional. Si esta presencia de funciones complejas resulta en una CPU que excede los límites del chip o de la plaqueta, la degradación en rendimiento puede ser aún peor.

Diseño Load/Store, Registro a Registro.

La mayoría de las instrucciones de las máquinas RISC involucran operaciones entre registros, estando sólo permitido a las instrucciones de LOAD y STORE el acceso a memoria. Esto permite por un lado simplificar el set de instrucciones y por lo tanto también la lógica de control. Por otro lado las arquitecturas RISC al motivar el uso de registros, permite que los operandos que son accedidos frecuentemente, permanezcan en un área de alta velocidad.

Control por Hardware.

La estructura de la unidad de control en un procesador de VLSI refleja claramente el set de instrucciones. La lógica necesaria para descodificar una instrucción compleja retarda el tiempo de descodificación, lo cual tiene un impacto en el rendimiento cuando la unidad de control esta en el camino crítico.

La simplicidad del control por hardware en los diseños RISC hace que solo el 6 ó 10 % del área del chip constituya funciones de control, mientras que en otros diseños (68000 o z8000) el área de control ocupa en un 50 y un 60 %. Esto ha permitido no sólo simplificar el diseño del chip, reduciendo el tiempo de diseño, sino también aprovechar el área no usada en funciones de control para destinarla principalmente a registros.

Instrucciones y Modos de Direccionamiento.

Las máquinas RISC se caracterizan por su reducido número de instrucciones y por estar compuestas por aquellas de mayor frecuencia de uso relativa. Los diseños RISC tienen entre 31 y 39 instrucciones para RISC I Y II, contra alrededor de 300 instrucciones de una máquina CISC como la VAX. El tener un set de instrucciones reducido permite simplificar la lógica de control.

Con respecto a los modos de direccionamiento, los diseños RISC usan los modos de direccionamiento más simples. En caso de ser necesario, los modos más complejos pueden ser sintetizados en software a través de los más simples. El usar los modos de direccionamiento más simples permite simplificar el set de instrucciones y en consecuencia la lógica de control.

Formato fijo de Instrucciones.

Una característica común a todos los diseños RISC, es justamente el tener un formato fijo de instrucciones. El formato fijo ayuda a simplificar la implementación pues, permite una simple y rápida decodificación. Se puede entonces asignar cada campo de la instrucción a una función particular, adecuada para un controlador por hardware, pudiendo evitarse microcódigos, lo que redundaría en una mayor velocidad.

La mayoría de los diseños RISC tienen instrucciones de 32 bits de longitud, lo que permite un direccionamiento real de 4 Gbytes de memoria.

pipelining.

Ya hemos visto que una forma de mejorar el rendimiento de una computadora es el uso de ejecución en pipeline, esta técnica es adoptada con diferentes variantes por todos los diseños RISC.

Maximización del uso de Registros.

El uso de procedimientos trae aparejado dos operaciones que involucran un alto consumo de tiempo: el almacenar o recuperar registros en cada call o return, y el pasar parámetros y resultados entre procedimientos. Este tiempo puede reducirse con técnicas que administren el uso de los registros.

Los diseñadores de las máquinas RISC encararon la forma de maximizar el uso de los registros desde dos ópticas totalmente diferentes. desde el punto de vista del compilador o desde el hardware.

Un ejemplo típico del primer caso es el algoritmo de coloración de grafos. La solución que se aplica en este caso son las reglas prescritas para pintar un grafo con un número fijo de colores, donde cada color representa uno de los registros.

En la siguiente figura se presentan las variables A a G durante el periodo de uso.

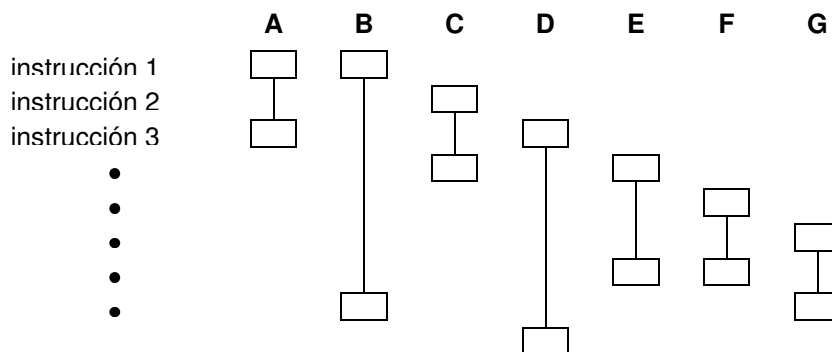
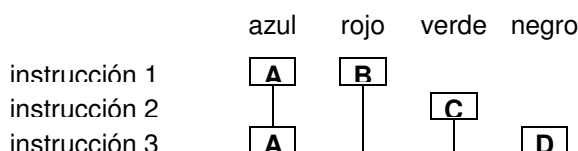


Figura 8-17.

Suponiendo que solo se dispone de 4 registros, el problema se reduce a encontrar la forma de mapear las 7 variables con los cuatro colores, lo que es equivalente a pintar el grafo con solo cuatro colores. Una solución a este problema puede ser la siguiente: Se comienza a mapear las primeras cuatro variables con los cuatro colores. Las variables **A** y **E** pueden compartir el mismo color ya que no tienen conflictos entre sí. Lo mismo ocurre con **C** y **F**. Para **G** en cambio, ya todos los colores están ocupados, de modo que el compilador debe usar LOADs y STOREs para liberar un registro y hacer espacio para **G**.

La solución implementada en hardware, en cambio, consiste en equipar al procesador con bancos múltiples de registros. El procesador asigna simplemente un nuevo banco de registros para cada llamada a procedimiento, evitando de esta manera el tener que almacenar los registros en memoria. Claro que el número de registros no es ilimitado. En muchos programas la profundidad del anidado de llamadas a procedimientos excederá el número de ventanas de registros suministradas por el procesador. Para estos casos se debe proveer un mecanismo que libere algunos registros moviendo su contenido a memoria.



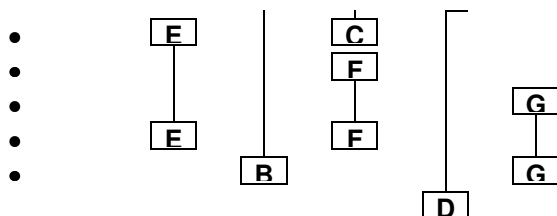


Figura 8-18

Ventanas de registros solapadas.

En función de su accesibilidad, los registros pueden ser clasificados en globales, locales y solapados. Los registros globales pertenecen a todas las ventanas y tienen los mismos números para cada ventana. Los registros locales pertenecen a una sola ventana. Los registros solapados pertenecen a dos ventanas pero tiene diferente número para cada una de ellas.

El tiempo de ejecución de un ciclo esta dado por el tiempo que lleva leer y sumar dos registros y guarda el resultado en otro registro.

Investigaciones realizadas en el uso de lenguajes de alto nivel indican que los llamados a procedimientos son los que ocupan mayor tiempo de ejecución. En el caso del RISC tendremos muchos de ellos ya que las instrucciones complejas de CISC son subrutinas de RISC. Por lo tanto en este caso el llamado a procedimiento debe ser más rápido que el usual. Para ello RISC ha utilizado el concepto de ventanas de registros con la ventaja adicional de reducir el tráfico de memoria.

Cada llamado a procedimiento aloja una nueva ventana de registros, extraída del archivo general de registros, para ser utilizada en este procedimiento; al ejecutar el retorno se resetea un indicador que restaura la ventana o conjunto anterior.

Cabe destacar que NO todos los registros se salvan o restauran durante los procedimientos; esos registros ($r_0 - r_9$) se llaman globales.

El conjunto de registros usado por cada procedimiento se superpone, de esta manera el pasaje de parámetros es automático.

Podemos ver entonces que para la selección de un registro determinado se necesitan dos parámetros: saber de que ventana se trata y seleccionar un registro dentro de dicha ventana. En RISC II la identificación de la ventana la suministra el estado de la CPU y la selección del registro viene especificada en la instrucción.

La ventana de parámetros tiene la estructura indicada en la siguiente figura:

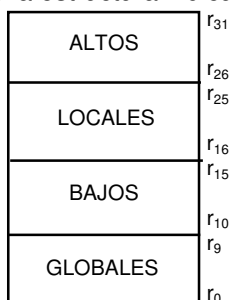


Figura 8-19

Los registros 'ALTOS' ($r_{26} - r_{31}$) contienen los parámetros transferidos por el procedimiento llamante. Los registros 'LOCALES' ($r_{16} - r_{25}$) se utilizan para el almacenamiento de variables locales. Los registros 'BAJOS' ($r_{10} - r_{15}$), aparecen en los registros 'ALTOS' ($r_{26} - r_{31}$) de la nueva ventana. Un encadenamiento de procedimientos A, B y C lo ejemplificamos en la siguiente figura 8-20.

El número de ventana cambia cada vez que se ejecuta un llamado a procedimiento. Por lo tanto, almacenar o recuperar registros cuando hay calls o returns no es necesario. Las variables locales que no son escalares son almacenadas en la memoria principal.

Las ventanas se hallan organizadas en stack, en el cual a los pares de procedimientos padre e hijo se les asigna ventanas solapadas adyacentes.

Preparándose para un posible llamamiento a un procedimiento, el padre escribe los argumentos actuales en los registros bajos de la ventana corriente y el hijo los tiene disponibles en los registros altos de su propia ventana. De esta forma el solapado de ventanas permite el pase de argumentos dentro de registros. Similarmente, el uso de ventanas solapadas permite el retorno de valores desde un procedimiento hijo a su padre.

Al tener las ventanas y las regiones de solapamiento un tamaño fijo, la decodificación se realiza simplemente por medio de una lógica AND-OR.

RISC II tiene 138 registros de 32 bits, de los cuales 10 registros son globales, estando el resto organizado en 8 ventanas de 6 registros solapados, 10 registros locales, y otros 6 registros solapados.

Las ventanas están organizadas en forma cíclicas (la ultima relacionada con al primera) y manejadas mediante el uso de dos punteros que llevan la cuenta de las ventanas ocupadas y vacías. El

Current Window Pointer (CWP) apunta a la ventana del procedimiento que está actualmente activo el Saved Window Pointer (SWP) identifica la última ventana que ha sido salvada en memoria.

Las variables guardadas en los registros solapados son visibles solamente por el procedimiento *hijo* en el lenguaje de alto nivel. Este procedimiento tiene un nombre para estas variables, pudiendo hacer referencia a las mismas. En el procedimiento *padre* en cambio, estos ítems no son variables sino valores de expresiones. Ésto es verdadero si los argumentos son pasados por valor. En el caso que los mismos sean pasados por referencia, los registros solapados contienen punteros que apuntan a los argumentos.

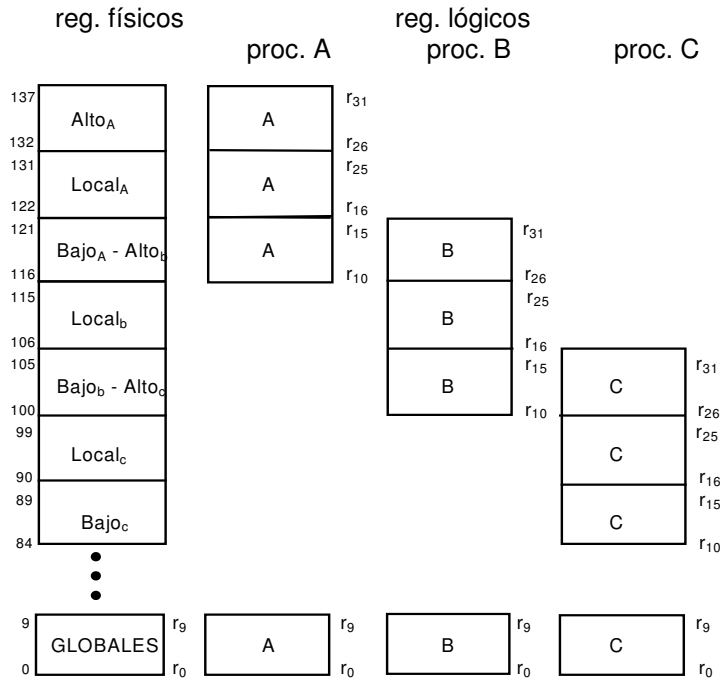


Figura 8-20.

Por ejemplo si el procedimiento D quiere llamar al procedimiento E, primero escribe los argumentos de E en los registros solapados comunes a ambos y luego ejecuta un call. El CWP se corre una ventana. Ahora si el procedimiento E quisiera llamar al procedimiento F, los argumentos de salida de F destruirían a los argumentos de entrada de A, lo que debemos evitar. Esto se logra comparando CWP con SWP. Cuando CWP es igual a SWP más 1, y se ejecuta una instrucción de call, se genera un trap de overflow que evita se complete la instrucción de call.

Lo que hace el trap es trasladar el control a una rutina que maneja el overflow. Esta rutina salva una o más ventanas en memoria y actualiza el SWP. En caso similar se procede en el caso de underflow de registros a raíz de returns.